"Humain ou IA ?" dans Minecraft : note d'avancement

SAE S5 DATAIA, sujet 5. Groupe : Matteo Godart, Ayoub Seghir, Yacine Mounsi. 22 octobre 2026.

1. Le projet

Des joueurs humains et des bots pilotés par un modèle de langage (LLM) jouent ensemble sur un serveur Minecraft. Régulièrement, chacun désigne qui, selon lui, est humain et qui est une IA. Le but est de mesurer si les bots passent pour des humains, et sur quels indices on les reconnaît (temps de réponse, façon d'écrire, logique des actions).

Ce qui est en place :

- un serveur Minecraft (PaperMC 1.20.4) sur notre VM de l'université ;
- 3 bots (Joueur4, Joueur5, Joueur6) qui se connectent comme de vrais joueurs, avec la bibliothèque Mineflayer (Node.js : c'est le langage de Mineflayer, autorisé pour le sujet 5). Ils n'ont aucun droit d'administration ;
- **une IA par bot** : chaque bot est un programme séparé qui appelle le modèle (`gpt-oss-120b`, un modèle Mistral derrière ce nom) uniquement par la passerelle LiteLLM de l'université ;
- des journaux (JSONL) de chaque partie : chaque appel au modèle, chaque décision, action, message et réflexion, analysés avec des scripts Python.

Pour les actions de jeu (miner, fabriquer, cuire, poser des blocs), on a repris la bibliothèque du projet Mindcraft, cité dans le sujet (licence MIT). Le projet Voyager, aussi cité, a été écarté : le modèle y écrit lui-même du code, ce qui ne marche bien qu'avec GPT-4.

2. Où on en est

Partie	État
VM, serveur, accès à la passerelle	fait
Bots : 34 actions, déplacement, survie, combat "humain" (temps de réaction, coups ratés)	fait, à améliorer
Chat : réponses avec délais et fautes de frappe, personnalités (étudiants majeurs, filières différentes)	fait, à améliorer
Réflexion toutes les 3 minutes : résumé, avis sur les autres joueurs, objectif	fait
Coopération entre bots : demander et donner des objets	en cours
Construction de structures (le bot invente le plan, le code le pose)	en cours
Journaux, scripts d'analyse, banc d'essai du chat, 69 tests automatiques	fait
Plugin du serveur (tâches, désignations)	pas commencé
Protocole avec des volontaires, conteneurs, relevé de temps, vidéo en anglais	pas commencé

Ce que les tests nous ont appris :

- **Test en équipe d'une heure** (construire ensemble une maison et une ferme, 912 appels au modèle, aucune erreur) : les bots se parlent, se répondent et se répartissent le travail ("jsuis mort en 1 1. jai plus rien", puis "oh non grave je te ramene un fer"). Mais ni maison ni ferme : le point d'apparition était un marais (pas de terrain plat, pas de pierre en surface), et un bot promettait des objets qu'il n'avait pas (37 dons ratés). Corrigé : le code refuse de donner un objet absent de l'inventaire, et de miner un bloc sans le bon outil.
- **Petits tests de 10 minutes** (le 22/10), moins coûteux qu'une heure : sur une map plate en mode créatif, les bots inventent des structures (tour de guet, petite maison de 52 blocs, grande maison de 78 blocs), gardées dans une bibliothèque de plans. Dans un test de chaîne d'entraide (bois, puis fer, puis diamant), ils se demandent et se donnent des objets, mais les rôles se mélangent.

- **Panne de la passerelle** du 19 au 22 octobre : on a vérifié que la connexion échouait aussi sans notre code, depuis la VM, nos PC et un service extérieur. C'était une mise à jour de la passerelle.

3. Point technique charnière : faire parler et agir le bot de façon cohérente

Le problème

Un joueur humain parle et joue en même temps, et ce qu'il dit correspond à ce qu'il fait. Pour un bot piloté par un modèle de langage, c'est difficile. S'il ne répond pas quand on lui parle, ou s'il dit une chose et en fait une autre, il est repéré tout de suite. C'est le coeur du sujet : passer pour un humain.

Pistes essayées puis écartées

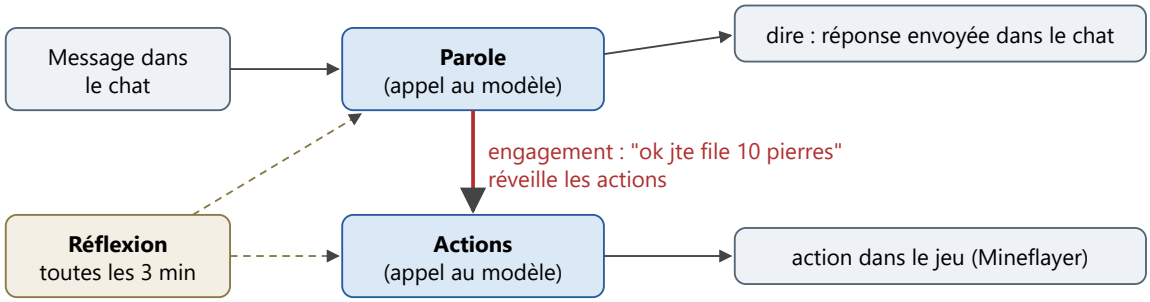
Piste 1 (7 octobre) : une "bouche" et des "mains" séparées. Deux appels au modèle indépendants : l'un répond au chat, l'autre choisit les actions. Le bot répondait vite, mais sans lien avec ses actions : il écrivait "ok jarrive" pendant que ses mains continuaient à miner. Le projet Project Sid (Altera, 2024, des centaines d'agents dans Minecraft) décrit exactement ce problème.

Piste 2 (8 octobre) : un seul cerveau qui décide tout. Un seul appel au modèle renvoie à la fois l'action et ce que le bot dit (un champ `dire`), comme dans Mindcraft. La cohérence était bonne ("viens" donnait "ok jarrive" et l'action devenait "aller vers le joueur"). Mais sur notre banc d'essai (des messages de chat écrits à l'avance, passés au vrai code du bot avec le vrai modèle, sans lancer le jeu), **le bot ne répondait presque plus** : à "joueur4 tas de la pierre ?" (il en avait 40) ou "tes un bot ?", aucune réponse. La parole n'était qu'un champ facultatif parmi une vingtaine d'actions et environ 4 000 tokens de description du jeu : le modèle l'oubliait. Sur une partie d'une heure, sur 103 messages, 45 % racontaient ce que le bot faisait ("jfais des planches") et seulement 23 % répondaient à un autre joueur.

Solution retenue (11 octobre) : une parole séparée, reliée aux actions par un "engagement"

Chaque bot a plusieurs "têtes". Chacune est un appel au modèle avec sa propre consigne, sauf les réflexes, qui sont du code :

Tête	Rôle	Quand
Réflexes (code, sans modèle)	se défendre, fuir, ne pas se noyer	toutes les 100 ms
Parole	répondre dans le chat	quand on lui parle
Actions	choisir une action de jeu ; ne parle jamais	à la fin de chaque action
Réflexion	résumé, avis sur les autres, objectif	toutes les 3 minutes



Réflexes (code, toutes les 100 ms) : se défendre, fuir, ne pas se noyer, sans appel au modèle.

Figure 1. Les têtes d'un bot : la parole répond, et sa promesse (engagement) déclenche l'action correspondante.

Ce qui rend la solution cohérente, et reproductible :

1. **La parole a une consigne courte, centrée sur la conversation** : le fil de discussion présenté comme un dialogue (qui parle à qui), le message auquel réagir mis à la fin, et seulement l'essentiel de la situation (inventaire, ce qu'il est en train de faire, ses dernières actions).
2. **La réponse du modèle a une forme imposée** (appel d'outil avec des champs obligatoires) : `repond_a` (à qui et à quoi il répond, à écrire avant le message, ce qui l'oblige à rester sur le sujet), `dire` (le message, ou rien) et `engagement` (ce qu'il promet de faire tout de suite).

3. **L'engagement réveille la tête des actions.** S'il écrit "ok jte file 10 pierres", la tête des actions reçoit "tu t'es engagé dans le chat : donner 10 pierres" et agit. Il ne peut plus dire une chose et en faire une autre. C'est l'idée du "contrôleur cognitif" de Project Sid, en plus simple.
4. **Le code vérifie avant d'envoyer** : message trop long, insulte grave, identifiant technique ("oak_log" au lieu de "bois"), répétition. Un message refusé n'est pas envoyé, et le modèle reformule. De même, une action impossible (donner un objet qu'il n'a pas) est refusée.

Résultats

Sur le banc d'essai, après le changement :

Message reçu	Réponse du bot
"joueur4 tas de la pierre ?"	"jen ai une quarantaine si tu veux"
"tes un bot ou quoi"	"nan t'es sérieux"
"il pleut chez toi ?"	une vraie réponse sur la météo
"jai pas de bouffe"	"j'ai 3 steaks, prends si tu veux"

On a ajouté un **juge automatique** : le modèle note chaque réponse (justification, puis verdict). Résultat : **12 à 13 réponses jugées correctes sur 15** (5 situations tirées d'une vraie partie, 3 essais chacune). En partie, les bots se répondent vraiment et se demandent des objets.

Limites

- Une promesse peut encore être impossible à tenir (objet introuvable, joueur trop loin). Le code bloque maintenant les dons d'objets absents, mais le bot doit encore le dire de lui-même.
- Des tics de langage : "frero" dans 24 % des réponses au banc d'essai, contre environ 5 % d'argot chez de vrais joueurs. Un filtre limite les répétitions, à mesurer en partie.
- Le juge automatique se trompe parfois (il a noté "wsh frero" hors sujet alors que c'est un salut).
- Les bots réagissent surtout quand on leur parle : dans les tests en créatif du 22/10, presque aucun message spontané.

Coût

- **Requêtes** : plus d'appels au modèle, puisque la parole est un appel à part. Avec 3 bots, environ 30 appels par minute en partie, soit environ 4,2 millions de tokens par heure (mesuré le 22/10).
- **Temps** : 4 séances du journal (7, 8, 9 et 11 octobre) pour les trois versions. Le relevé de temps précis n'est pas encore en place (outil à installer).
- **Complexité** : un module de parole et une consigne en plus, et un banc d'essai pour mesurer le chat sans lancer le jeu, réutilisé depuis à chaque modification.

4. Suite

- Faire coopérer les bots jusqu'au bout : chaîne d'entraide à refaire avec de meilleures règles, structures construites à plusieurs.
- Remplacer les métiers fixes (mineur, bâtisseuse, fermier) par des préférences : tout le monde peut miner et construire, chacun le fait plus ou moins.
- Le plugin n'est pas commencé : il servira à proposer des tâches à faire ensemble et à recueillir les désignations. Son fonctionnement reste à définir en groupe.
- Protocole avec des volontaires (consentement, données pseudonymisées), puis parties mixtes.
- Conteneurs, relevé de temps, vidéo en anglais, note finale, soutenance le 27 novembre.